

Practical_07:-

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Output for Practical_07:-

```
v_santosh@DESKTOP-E34M7RI:~/Practical/Practical_07.c$ nano Practical_07.c
v_santosh@DESKTOP-E34M7RI:~/Practical/Practical_07.c$ gcc Practical_07.c -o Practical_07
v_santosh@DESKTOP-E34M7RI:~/Practical/Practical_07.c$ ./Practical_07
===== Memory Address Analysis =====
Process PID          : 637

Code (Text) Segment  :
  Function Address    : 0x589ae2da3229

Data Segment (Initialized):
  Global variable     : 0x589ae2da6010
  Static variable     : 0x589ae2da6014

BSS Segment (Uninitialized):
  Global variable     : 0x589ae2da601c
  Static variable     : 0x589ae2da6020

Heap Segment         :
  Allocated memory    : 0x589b05fe7010

Stack Segment        :
  Local variable      : 0x7ffcc4500984
=====

Paused to inspect memory maps.
Run 'cat /proc/637/maps' in another terminal.
Press ENTER to exit...
v_santosh@DESKTOP-E34M7RI:~/Practical/Practical_07.c$ |
```

Code for Practical_07:-

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

// Global variables
int global_initialized = 42;    // Initialized Data Segment (.data)
int global_uninitialized;      // Uninitialized Data Segment (.bss)

void print_addresses(void) {
    // Local static variables
    static int static_initialized = 100; // Data Segment (.data)
    static int static_uninitialized;      // BSS Segment (.bss)

    // Stack variable
    int stack_var = 10;

    // Heap allocation
    int *heap_var = (int *)malloc(sizeof(int));
    if (!heap_var) {
        perror("Allocation failed");
        return;
    }

    // Convert function pointer to standard void pointer safely
    void *code_ptr;
    *(void **)&code_ptr = (void *)print_addresses;

    printf("===== Memory Address Analysis =====\n");
    printf("Process PID      : %d\n", getpid());

    printf("Code (Text) Segment  : \n");
    printf("  Function Address   : %p\n", code_ptr);

    printf("Data Segment (Initialized): \n");
    printf("  Global variable    : %p\n", (void *)&global_initialized);
    printf("  Static variable    : %p\n", (void *)&static_initialized);

    printf("BSS Segment (Uninitialized): \n");
    printf("  Global variable    : %p\n", (void *)&global_uninitialized);
    printf("  Static variable    : %p\n", (void *)&static_uninitialized);

    printf("Heap Segment         : \n");
    printf("  Allocated memory   : %p\n", (void *)heap_var);

    printf("Stack Segment        : \n");
    printf("  Local variable     : %p\n", (void *)&stack_var);
    printf("===== \n");

    free(heap_var);
}

int main(void) {
    print_addresses();

    printf("\nPaused to inspect memory maps.\n");
    printf("Run 'cat /proc/%d/maps' in another terminal.\n", getpid());
    printf("Press ENTER to exit...");
    getchar();

    return 0;
}
```